

```
In [1]: %matplotlib ipynb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from scipy import stats
from statsmodels.stats.multicomp import (pairwise_tukeyhsd,
                                         MultiComparison)

from matplotlib.backends.backend_pdf import PdfPages
```

Groupby!

NYC Trees

We are going to look at NYC trees. There is one weird caveat. Since many answers have the word none we need to tell pandas that none is real and only blanks are NaN or not a number.

- read in the data as shown
- Then check the data like usual by doing
 - columns
 - info()
 - just looking at the dataframe.
 - then we can get started

```
In [115... filename='2015_Street_Tree_Census_-_Tree_Data_20250411.csv'
#df=pd.read_csv(filename,na_values='',keep_default_na=False)
df=pd.read_csv(filename,na_values='',keep_default_na=False)
```

```
In [104... df.columns
```

```
Out[104... Index(['tree_id', 'block_id', 'created_at', 'tree_dbh', 'stump_diam',
        'curb_loc', 'status', 'health', 'spc_latin', 'spc_common', 'steward',
        'guards', 'sidewalk', 'user_type', 'problems', 'root_stone',
        'root_grate', 'root_other', 'trunk_wire', 'trnk_light', 'trnk_other',
        'brch_light', 'brch_shoe', 'brch_other', 'address', 'postcode',
        'zip_city', 'community board', 'borocode', 'borough', 'cnclldist',
        'st_assem', 'st_senate', 'nta', 'nta_name', 'boro_ct', 'state',
        'latitude', 'longitude', 'x_sp', 'y_sp', 'council district',
        'census tract', 'bin', 'bbl'],
        dtype='object')
```

```
In [105... df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 683788 entries, 0 to 683787
Data columns (total 45 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tree_id                               683788 non-null  int64
1   block_id                              683788 non-null  int64
2   created_at                            683788 non-null  object
3   tree_dbh                              683788 non-null  int64
4   stump_diam                            683788 non-null  int64
5   curb_loc                              683788 non-null  object
6   status                                683788 non-null  object
7   health                                652172 non-null  object
8   spc_latin                             652169 non-null  object
9   spc_common                            652169 non-null  object
10  steward                               652173 non-null  object
11  guards                                652172 non-null  object
12  sidewalk                              652172 non-null  object
13  user_type                             683788 non-null  object
14  problems                              652124 non-null  object
15  root_stone                            683788 non-null  object
16  root_grate                            683788 non-null  object
17  root_other                            683788 non-null  object
18  trunk_wire                            683788 non-null  object
19  trnk_light                            683788 non-null  object
20  trnk_other                            683788 non-null  object
21  brch_light                            683788 non-null  object
22  brch_shoe                             683788 non-null  object
23  brch_other                            683788 non-null  object
24  address                               683788 non-null  object
25  postcode                              683788 non-null  int64
26  zip_city                              683788 non-null  object
27  community board                       683788 non-null  int64
28  borocode                              683788 non-null  int64
29  borough                               683788 non-null  object
30  cncldist                              683788 non-null  int64
31  st_assem                              683788 non-null  int64
32  st_senate                             683788 non-null  int64
33  nta                                   683788 non-null  object
34  nta_name                             683788 non-null  object
35  boro_ct                               683788 non-null  int64
36  state                                 683788 non-null  object
37  latitude                              683788 non-null  float64
38  longitude                             683788 non-null  float64
39  x_sp                                  683788 non-null  float64
40  y_sp                                  683788 non-null  float64
41  council district                      677269 non-null  float64
42  census tract                          677269 non-null  float64
43  bin                                   674229 non-null  float64
44  bbl                                   674229 non-null  float64
dtypes: float64(8), int64(11), object(26)
memory usage: 234.8+ MB

```

In [106... df

Out [106...

	tree_id	block_id	created_at	tree_dbh	stump_diam	curb_loc	status	health	spc_lat	spc_cor
0	180683	348711	08/27/2015	3	0	OnCurb	Alive	Fair	Acer rubrum	red
1	200540	315986	09/03/2015	21	0	OnCurb	Alive	Fair	Quercus palustris	p
2	204026	218365	09/05/2015	3	0	OnCurb	Alive	Good	Gleditsia triacanthos var. inermis	honey
3	204337	217969	09/05/2015	10	0	OnCurb	Alive	Good	Gleditsia triacanthos var. inermis	honey
4	189565	223043	08/30/2015	21	0	OnCurb	Alive	Good	Tilia americana	Am
...	...	...	...	...	...	...	...	...	...	...
683783	155433	217978	08/18/2015	25	0	OnCurb	Alive	Good	Quercus palustris	p
683784	183795	348185	08/29/2015	7	0	OnCurb	Alive	Good	Cladrastis kentukea	Ker yellow
683785	166161	401670	08/22/2015	12	0	OnCurb	Alive	Good	Acer rubrum	red
683786	184028	504204	08/29/2015	9	0	OnCurb	Alive	Good	Acer rubrum	red
683787	200607	306527	09/03/2015	23	0	OnCurb	Alive	Fair	Acer rubrum	red

683788 rows x 45 columns

In [107...

```
df.describe()
```

Out [107...

	tree_id	block_id	tree_dbh	stump_diam	postcode	community board
count	683788.000000	683788.000000	683788.000000	683788.000000	683788.000000	683788.000000
mean	365205.011085	313793.096236	11.279787	0.432463	10916.246044	343.505404
std	208122.092902	114839.024312	8.723042	3.290241	651.553364	115.740601
min	3.000000	100002.000000	0.000000	0.000000	83.000000	101.000000
25%	186582.750000	221556.000000	4.000000	0.000000	10451.000000	302.000000
50%	366214.500000	319967.000000	9.000000	0.000000	11214.000000	402.000000
75%	546170.250000	404624.000000	16.000000	0.000000	11365.000000	412.000000
max	722694.000000	999999.000000	450.000000	140.000000	11697.000000	503.000000

Have Gemini Help us

- I have been trying to use gemini with google to help my programming.
- Gemini can't handle this whole dataset.
- Make a smaller dataset for 50000 random data points
- just call sample and save it to a csv and now you can upload it to gemini
- This will randomly choose a subset of the data

- We can come back for help when needed
- but you have a smaller dataset you can upload if needed

```
In [108... df.sample(n=50000).to_csv('small_trees.csv')
```

Start simple

- Where are the trees?
- how many trees per borough
- table and a bar plot

good ole value_counts!

```
In [123... df['borough'].value_counts()
```

```
Out[123... borough
Queens      250551
Brooklyn    177293
Staten Island 105318
Bronx       85203
Manhattan   65423
Name: count, dtype: int64
```

as a percentage

```
In [124... df['borough'].value_counts(normalize=True)*100
```

```
Out[124... borough
Queens      36.641620
Brooklyn    25.928065
Staten Island 15.402142
Bronx       12.460441
Manhattan   9.567732
Name: proportion, dtype: float64
```

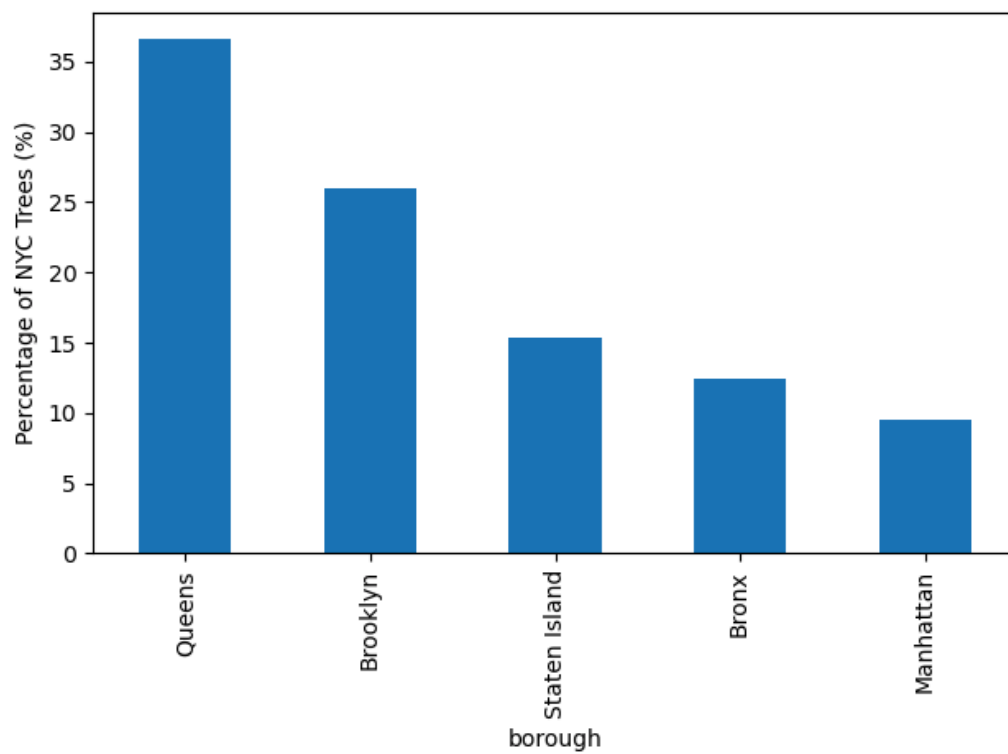
As a bar plot

```
In [178... fig,ax=plt.subplots(layout='tight')

(df['borough'].value_counts(normalize=True)*100).plot.bar()
ax.set_ylabel('Percentage of NYC Trees (%)')
```

```
Out[178... Text(0, 0.5, 'Percentage of NYC Trees (%)')
```

Figure



The obvious next step is to map the trees

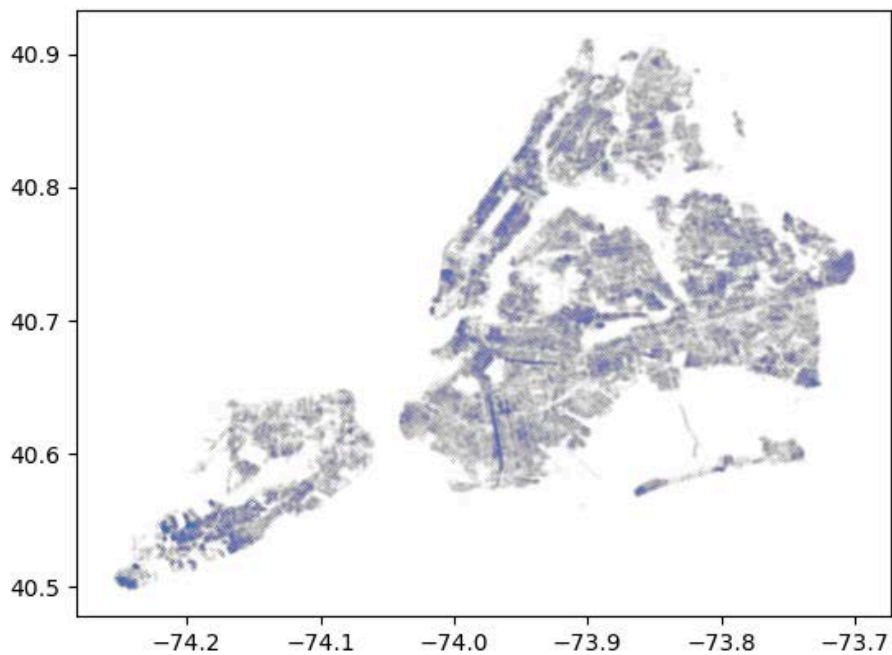
We won't make a real map as it is not the point this week.

- But we can do a quick map.
- Just make a scatter of the points use lat,long (reverse them)
- make the points really really small since there are so many

```
In [130... fig,ax=plt.subplots()  
ax.scatter(df['longitude'],df['latitude'],s=.00001)
```

```
Out[130... <matplotlib.collections.PathCollection at 0x145794150>
```

Figure



nice! moving on!

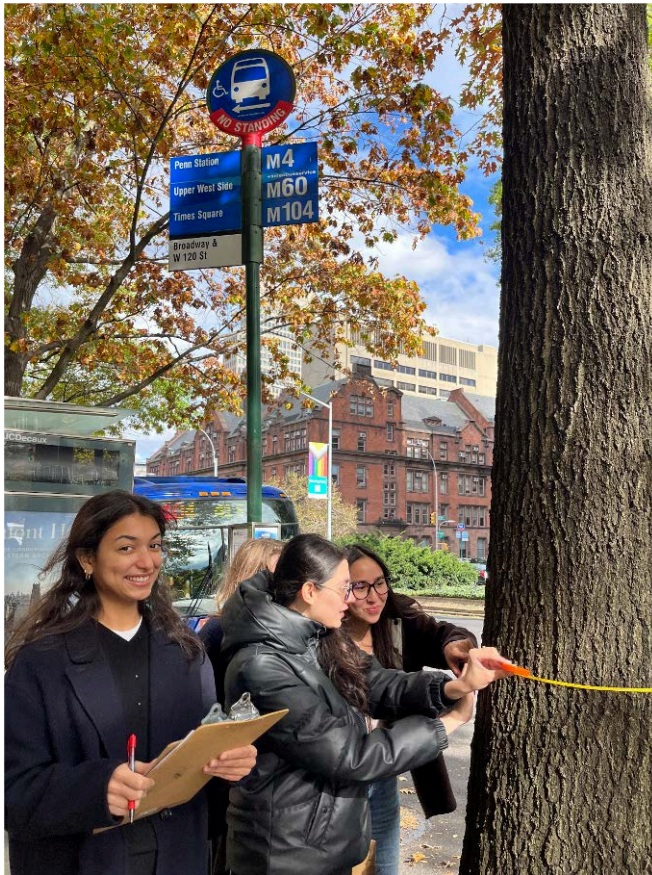
In []:

Tree size

- We are interested in tree size.
- this is dbh which is diameter at breast height
- the picture shows students measuring DBH
- let's start by just describing that data

```
In [6]: from IPython.display import Image
        Image('DBH_students.png',width=400)
```

Out [6]:



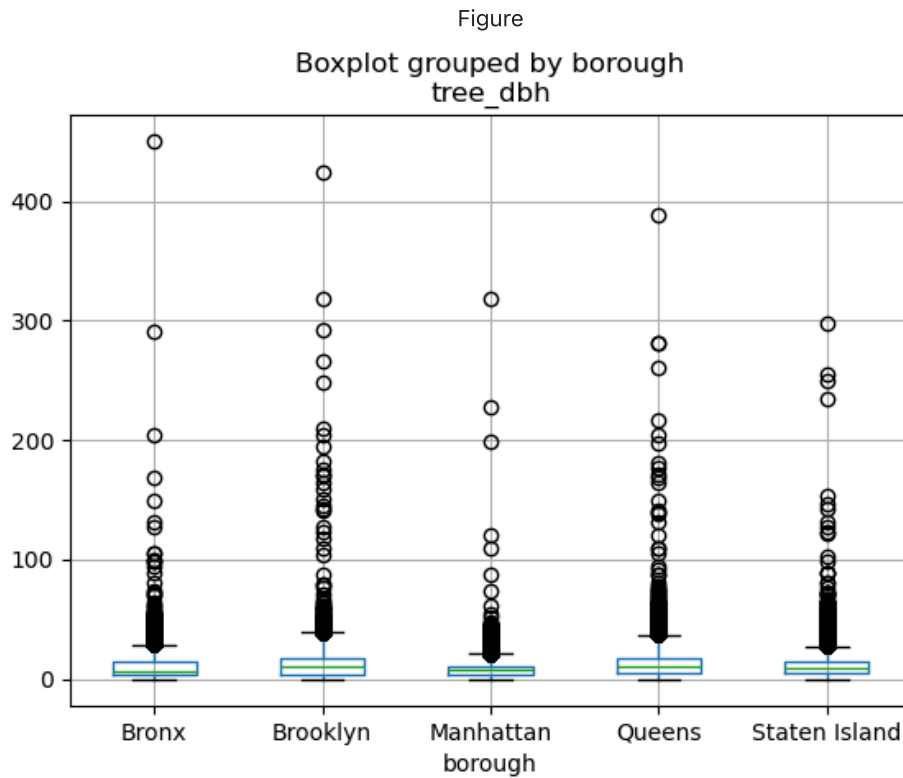
```
In [117...] df['tree_dbh'].describe()
```

```
Out[117...] count    683788.000000
mean         11.279787
std           8.723042
min           0.000000
25%           4.000000
50%           9.000000
75%          16.000000
max          450.000000
Name: tree_dbh, dtype: float64
```

Now make your boxplot of DBH by borough

```
In [126...] fig,ax=plt.subplots()
df.boxplot(column='tree_dbh',by='borough',ax=ax)
```

```
Out[126...] <Axes: title={'center': 'tree_dbh'}, xlabel='borough'>
```

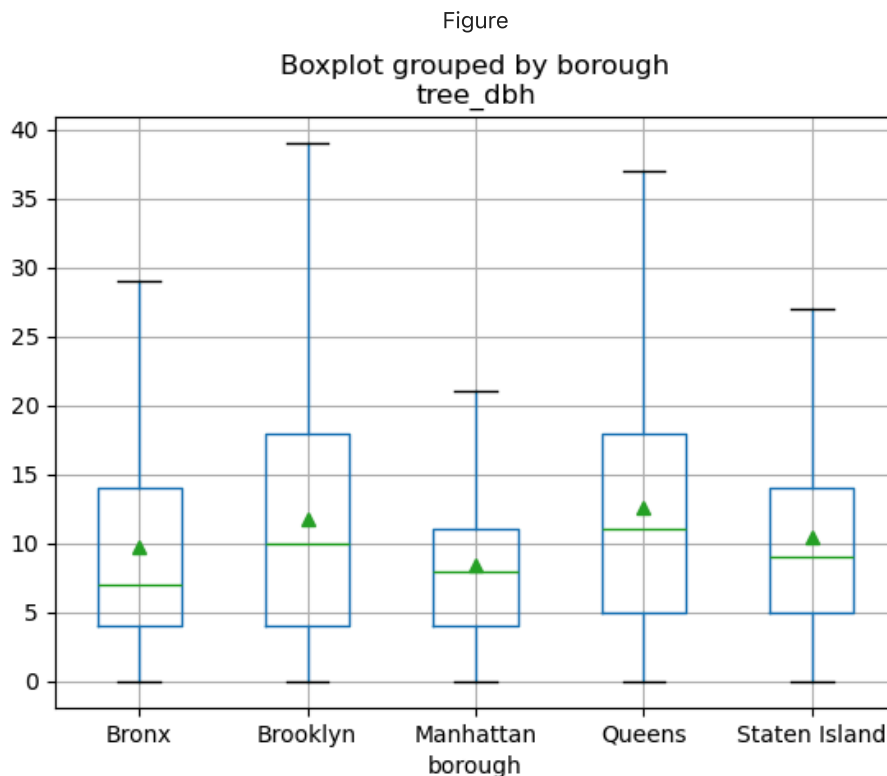


Now ask gemini to do it

- if it uses seaborn ask it to redo it with pandas
- if it does it with plt ask it to use `fig,ax=plt.subplots()`
- ask it how to remove outliers and show means
- `showfliers=False`
- `showmeans=True`

```
In [129... fig,ax=plt.subplots()
df.boxplot(column='tree_dbh',by='borough',ax=ax,showfliers=False,showmeans=True)
```

```
Out[129... <Axes: title={'center': 'tree_dbh'}, xlabel='borough'>
```

Now ask gemini how you turn the dbh by borough boxplot into a table.

It will says use groupby and describe.

So lets go learn this

Now what if we actually want the data.

- use the groupby function
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html>
- Here is how it works
 - Choose your columns you want to analyze
 - Choose the column you want to groupby
 - choose how you want to aggregate <https://stackoverflow.com/questions/53943319/what-are-all-pandas-agg-functions>
- Example
 - want to analyze DBH
 - Want to groupby borough
 - want to aggregate by mean.

```
In [70]: df.groupby(by=['borough'])['tree_dbh'].mean()
```

```
Out[70]: borough
Bronx      9.693649
Brooklyn   11.738884
Manhattan   8.473641
Queens     12.557870
Staten Island 10.492746
Name: tree_dbh, dtype: float64
```

You can do mor than just mean. You can even make your own functions. Use describe to get all the parameters

```
In [71]: df.groupby(by=['borough'])['tree_dbh'].describe()
```

```
Out[71]:
```

	count	mean	std	min	25%	50%	75%	max
borough								
Bronx	85203.0	9.693649	7.919802	0.0	4.0	7.0	14.0	450.0
Brooklyn	177293.0	11.738884	9.071056	0.0	4.0	10.0	18.0	425.0
Manhattan	65423.0	8.473641	5.658584	0.0	4.0	8.0	11.0	318.0
Queens	250551.0	12.557870	9.352517	0.0	5.0	11.0	18.0	389.0
Staten Island	105318.0	10.492746	8.062614	0.0	5.0	9.0	14.0	298.0

Now you can sort by the mean biggest to smallest

```
In [72]: df.groupby(by=['borough'])['tree_dbh'].describe().sort_values(by='mean',ascending=False)
```

```
Out[72]:
```

	count	mean	std	min	25%	50%	75%	max
borough								
Queens	250551.0	12.557870	9.352517	0.0	5.0	11.0	18.0	389.0
Brooklyn	177293.0	11.738884	9.071056	0.0	4.0	10.0	18.0	425.0
Staten Island	105318.0	10.492746	8.062614	0.0	5.0	9.0	14.0	298.0
Bronx	85203.0	9.693649	7.919802	0.0	4.0	7.0	14.0	450.0
Manhattan	65423.0	8.473641	5.658584	0.0	4.0	8.0	11.0	318.0

Now lets do some fun groupbying!

First let's look at guards by borough

- Want guards seperated by borough
- We will use groupby for this
- groupby with size gives you counts!

```
In [42]: Image('guards.jpg',width=400)
```

Out[42]:

In [142... `df.groupby(['borough', 'guards'], dropna=False).size()`

```
Out[142...
borough    guards
Bronx      Harmful      976
           Helpful     4605
           None       74524
           Unsure      480
           NaN        4618
Brooklyn    Harmful     5650
           Helpful    19471
           None     142563
           Unsure     2060
           NaN       7549
Manhattan   Harmful     8741
           Helpful    17559
           None     34812
           Unsure     1315
           NaN       2996
Queens      Harmful     3690
           Helpful     7153
           None    224356
           Unsure     2774
           NaN     12578
Staten Island Harmful     1195
           Helpful     3078
           None    96051
           Unsure     1119
           NaN      3875
dtype: int64
```

to make it into a nice table unstack it

In [144... `df.groupby(['borough', 'guards'], dropna=False).size().unstack()`

Out [144...

	guards	Harmful	Helpful	None	Unsure	NaN
borough						
Bronx		976	4605	74524	480	4618
Brooklyn		5650	19471	142563	2060	7549
Manhattan		8741	17559	34812	1315	2996
Queens		3690	7153	224356	2774	12578
Staten Island		1195	3078	96051	1119	3875

As a stacked bar plot

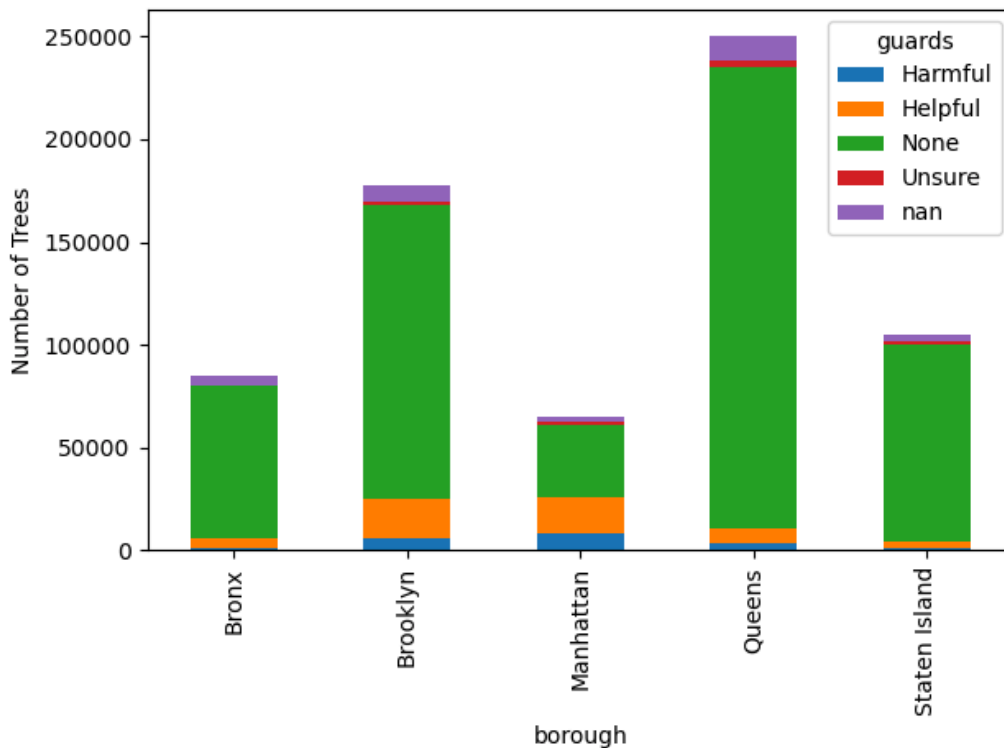
In [179...

```
fig,ax=plt.subplots(layout='tight')
df.groupby(['borough','guards'], dropna=False).size().unstack().plot.bar(ax=ax,stacked=True)
ax.set_ylabel('Number of Trees')
```

Out[179...

Text(0, 0.5, 'Number of Trees')

Figure



Turn into a percentage

- There are three ways to turn into a %
 - % of all #'s
 - % of the row
 - % of the column

To turn into a percent of everything divide by number of rows.

This works as we are counting all NaN's.

In [158...

```
(df.groupby(['borough','guards'], dropna=False).size()
 /df.shape[0]*100).unstack()
```

Out [158...

	guards	Harmful	Helpful	None	Unsure	NaN
borough						
Bronx	0.142734	0.673454	10.898700	0.070197	0.675356	
Brooklyn	0.826279	2.847520	20.849006	0.301263	1.103997	
Manhattan	1.278320	2.567901	5.091052	0.192311	0.438147	
Queens	0.539641	1.046084	32.810754	0.405681	1.839459	
Staten Island	0.174762	0.450140	14.046898	0.163647	0.566696	

To make the row add up to 100% divide by the groupby of the row

In [157...

```
(df.groupby(['borough', 'guards'], dropna=False).size()
 /df.groupby(['borough'], dropna=False).size()*100).unstack()
```

Out [157...

	guards	Harmful	Helpful	None	Unsure	NaN
borough						
Bronx	1.145500	5.404739	87.466404	0.563360	5.419997	
Brooklyn	3.186815	10.982385	80.410958	1.161918	4.257923	
Manhattan	13.360745	26.839185	53.210645	2.009996	4.579429	
Queens	1.472754	2.854908	89.545043	1.107160	5.020136	
Staten Island	1.134659	2.922577	91.200934	1.062496	3.679333	

You can quickly turn this into a bar chart

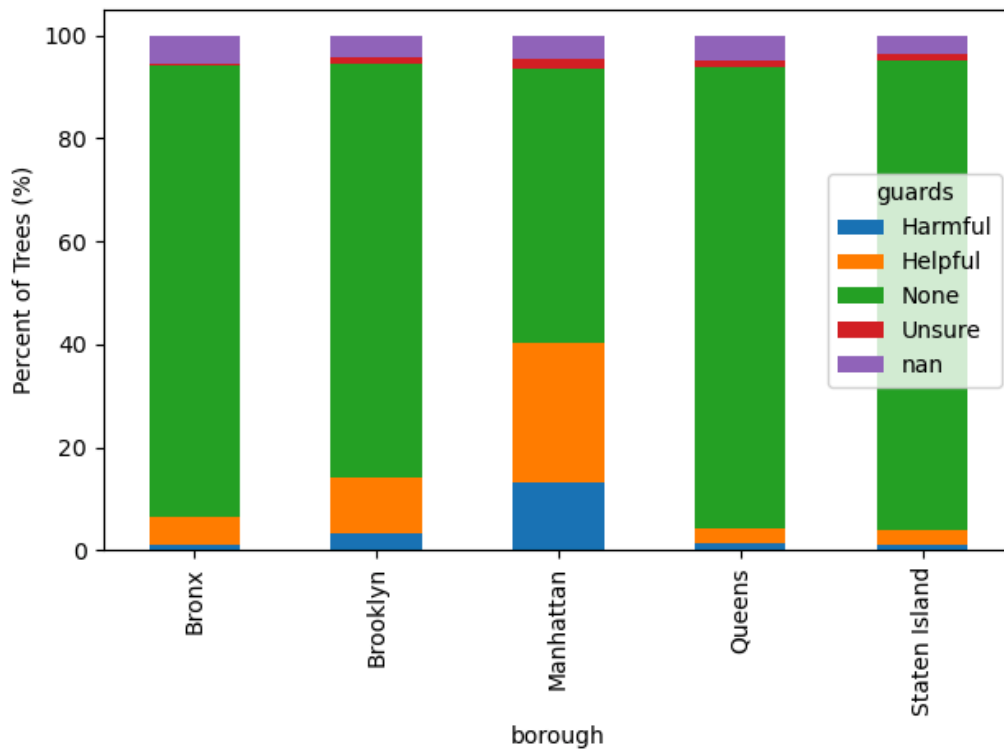
In [181...

```
fig,ax=plt.subplots(layout='tight')
(df.groupby(['borough', 'guards'], dropna=False).size()
 /df.groupby(['borough'], dropna=False).size()*100).unstack().plot.bar(ax=ax,stacked=True)
ax.set_ylabel('Percent of Trees (%)')
```

Out [181...

```
Text(0, 0.5, 'Percent of Trees (%)')
```

Figure



To make the columns add up to 100% divide by the groupby of the columns

```
In [182... (df.groupby(['borough', 'guards'], dropna=False).size()
/df.groupby(['guards'], dropna=False).size()*100).unstack()
```

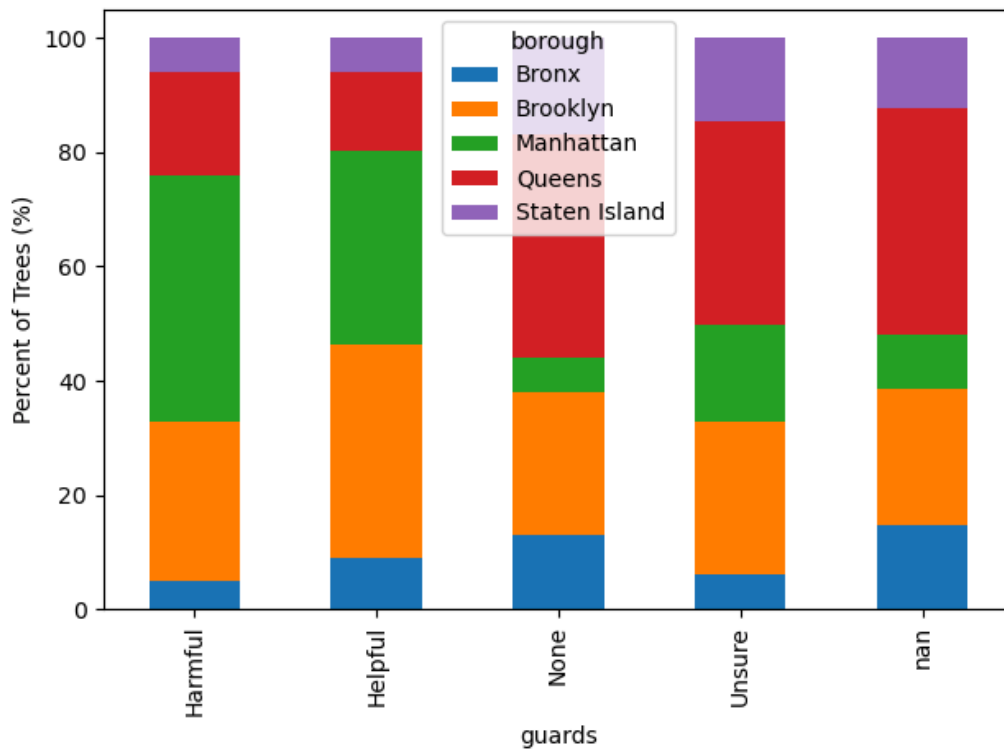
```
Out[182...
guards  Harmful  Helpful  None  Unsure  NaN
borough
Bronx      4.819277  8.878649  13.021705  6.195147  14.606528
Brooklyn  27.898479  37.540971  24.910275  26.587506  23.877151
Manhattan 43.161169  33.854548   6.082760  16.972122   9.476215
Queens    18.220423  13.791308  39.202105  35.802788  39.783654
Staten Island 5.900652  5.934524  16.783154  14.442437  12.256452
```

To turn this one into a bar chart you need to transpose it as you are summing over the columns. transposing flips the matrix

```
In [183... fig,ax=plt.subplots(layout='tight')
(df.groupby(['borough', 'guards'], dropna=False).size()
/df.groupby(['guards'], dropna=False).size()*100).T.plot.bar(ax=ax, stacked=True)
ax.set_ylabel('Percent of Trees (%)')
```

```
Out[183... Text(0, 0.5, 'Percent of Trees (%)')
```

Figure



In []:

Next question

- Are helpful guards associated with bigger or smaller trees?

```
In [166...] df.groupby(by=['guards'], dropna=False)['tree_dbh'].describe()
```

```
Out[166...]
count      mean      std  min  25%  50%  75%  max
guards
Harmful    20252.0  10.583646  6.080190  0.0   6.0  10.0  13.0  170.0
Helpful    51866.0   8.537886  6.267371  0.0   4.0   7.0  11.0  159.0
None       572306.0  12.058201  8.851880  0.0   5.0  10.0  17.0  425.0
Unsure      7748.0  10.125452  6.757314  0.0   5.0   9.0  13.0   53.0
NaN         31616.0   2.415992  5.050724  0.0   0.0   0.0   3.0  450.0
```

Trees with no guards are the biggest? Weird.... Maybe they only put guards around young or recent trees???
 Maybe just around trees in higher traffic areas? Not much difference between helpful or harmful guards.

Quick map!

- You can just make a scatter map of the location of the trees
- There are so many the outline of the city shows up
- We will not make a real map. If you really need to I could show you how to add a shapefile of the city.
- But we can make a dictionary of colors for our guards. I just want to see where any guards are. so I color both harmful and helpful red use a dict.
- follow the code and it is easy. You are just mapping it to a new parameter.

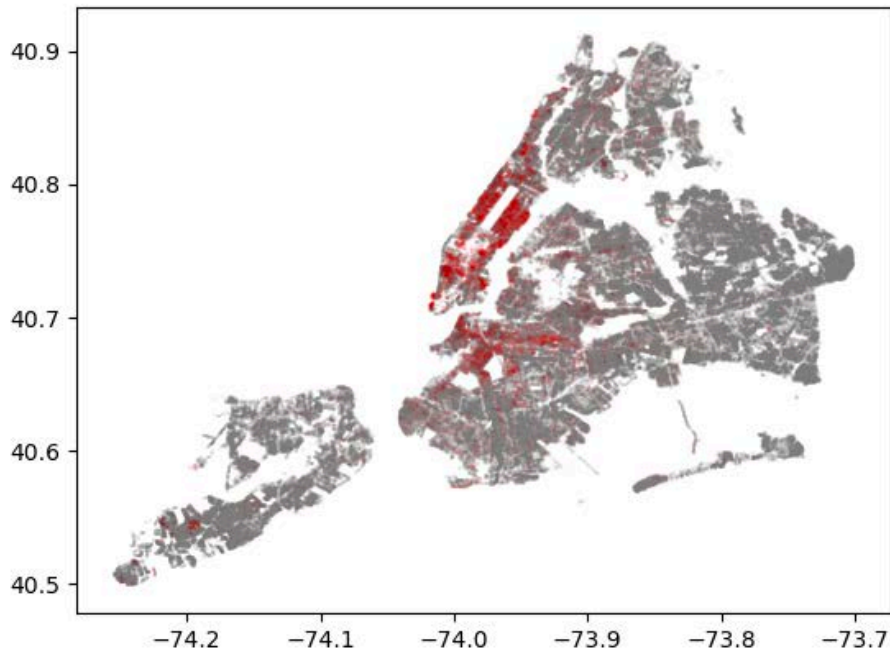
```
In [165... # Define colors for each category
category_colors = {'Harmful': 'Red', 'Helpful': 'Red', 'None': 'grey', 'Unsure': 'grey', np.nan: 'gre

# Create a list of colors for each data point based on its category
point_colors = [category_colors[cat] for cat in df['guards']]

fig, ax = plt.subplots()
ax.scatter(df['longitude'], df['latitude'], c=point_colors, s=.0001)
```

Out[165... <matplotlib.collections.PathCollection at 0x1572f7310>

Figure



In []:

you can add more items to groupby!

it gets confusing

```
In [162... df.groupby(['borough', 'guards', 'steward'], dropna=False).size().unstack()
```


Out [162...

	steward	1or2	3or4	4orMore	None	NaN
borough	guards					
Bronx	Harmful	525.0	60.0	5.0	386.0	NaN
	Helpful	3179.0	409.0	29.0	988.0	NaN
	None	10848.0	348.0	33.0	63295.0	NaN
	Unsure	256.0	38.0	4.0	182.0	NaN
	NaN	NaN	NaN	NaN	NaN	4618.0
Brooklyn	Harmful	3534.0	691.0	47.0	1378.0	NaN
	Helpful	12045.0	4261.0	402.0	2763.0	NaN
	None	27077.0	795.0	68.0	114623.0	NaN
	Unsure	1221.0	303.0	16.0	520.0	NaN
	NaN	NaN	NaN	NaN	NaN	7549.0
Manhattan	Harmful	5215.0	2156.0	84.0	1286.0	NaN
	Helpful	10498.0	5048.0	406.0	1607.0	NaN
	None	7773.0	233.0	20.0	26786.0	NaN
	Unsure	689.0	380.0	44.0	202.0	NaN
	NaN	NaN	NaN	NaN	NaN	2996.0
Queens	Harmful	2552.0	397.0	40.0	701.0	NaN
	Helpful	4655.0	1272.0	114.0	1112.0	NaN
	None	32864.0	1186.0	160.0	190146.0	NaN
	Unsure	1797.0	210.0	26.0	741.0	NaN
	NaN	NaN	NaN	NaN	1.0	12577.0
Staten Island	Harmful	800.0	118.0	6.0	271.0	NaN
	Helpful	1828.0	461.0	25.0	764.0	NaN
	None	15539.0	724.0	76.0	79712.0	NaN
	Unsure	662.0	93.0	5.0	359.0	NaN
	NaN	NaN	NaN	NaN	NaN	3875.0

remember if ever needed to_excel will dump things into an excel file

```
In [186... df.groupby(['borough', 'guards', 'steward'], dropna=False).size().unstack().to_excel('ExcelFile.xls')
```

Some more easy questions

most popular trees

What are the most populare trees by percentages of each tree?

```
In [74]: df['spc_common'].value_counts(normalize=True)*100
```

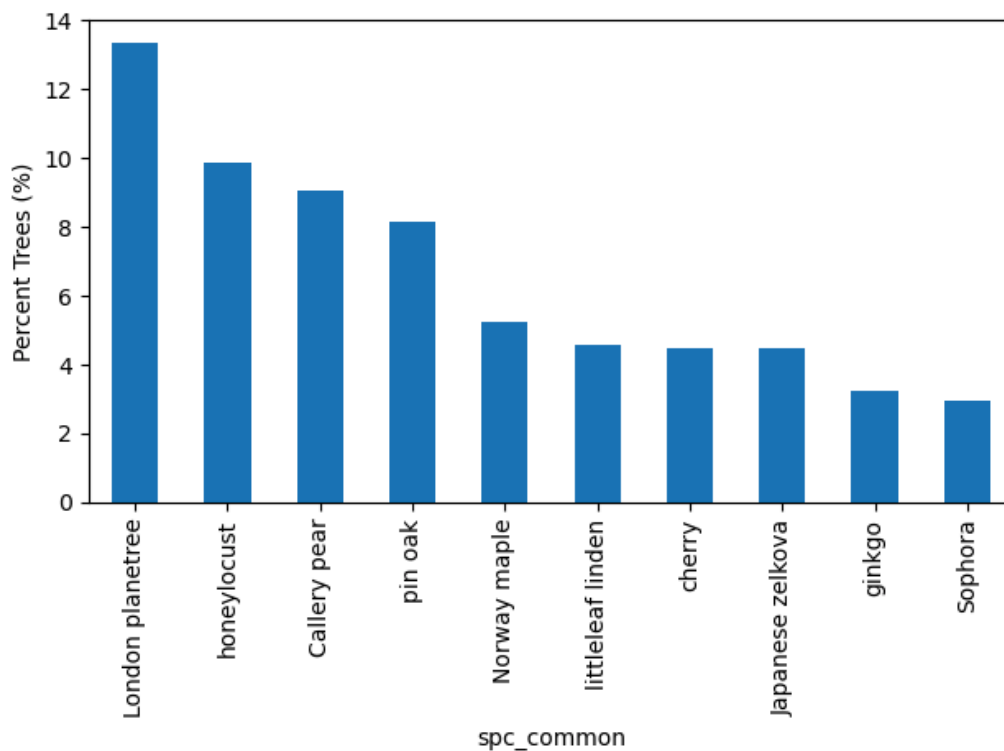
```
Out[74]: spc_common
London planetree    13.342247
honeylocust         9.853888
Callery pear        9.036155
pin oak             8.155095
Norway maple        5.242353
...
black pine          0.005673
pitch pine          0.005060
Osage-orange        0.004447
Scots pine          0.003833
Virginia pine       0.001533
Name: proportion, Length: 132, dtype: float64
```

You can easily turn this into a plot of the percentage of the top 10 trees

```
In [184... fig,ax=plt.subplots(layout='tight')
(df['spc_common'].value_counts(normalize=True)*100)[0:10].plot.bar(ax=ax)
ax.set_ylabel('Percent Trees (%)')
```

```
Out[184... Text(0, 0.5, 'Percent Trees (%)')
```

Figure



t-test

- two parameters!!!!
- is there a statistical difference.
- I am not going into the details. You can read and learn about t-tests
- This is the easiest way I found to do it.
 - I state which columns I want for data and grouping
 - Then you can use groupby to rearrange the data.
 - This is where it gets funky. The lambda function puts them in separate lists
 - Then the * passes them correctly
 - I can not find a good write up on how the * works (working on it) but it pulls out the data and passes it

- The p-value tells you the significanes.

```
In [44]: col_for_data='tree_dbh'
col_to_group='root_other'
stats.ttest_ind(*df.groupby(col_to_group)[col_for_data].apply(lambda x:x.values))
```

```
Out[44]: TtestResult(statistic=-63.79608098894526, pvalue=0.0, df=683786.0)
```

```
In [ ]:
```

Anova

If you want to see if the differences between the groups is significant you do an anova. If it is two groups you do a t-test. If it is more than two groups you do an Anova.

- One-way anova tells you if there is a significant difference among groups.
- it does not tell you which groups.
- Again it is the p-value we want!
- The stats.f_oneway needs a numpy array. so you use groupby to make the arrays. Then make separate columns
- then you run the stats.
- Then I print the stats
- This is the format samples = [cols[1] for cols in df.groupby(col_to_group)[col_for_data]]

```
In [45]: col_for_data='tree_dbh'
col_to_group='borough'

f_val, p_val = stats.f_oneway(*df.groupby(col_to_group)[col_for_data].apply(lambda x:x.values))
print('F value: {:.3f}, p value: {:.3f}\n'.format(f_val, p_val))
```

```
F value: 4178.336, p value: 0.000
```

Post-hoc Test

- We will not go in depth to Anovas
- An anova tells you if there is a difference. Not which pairs are different
- The post-hoc test tells you if the pairs are statistically different
- tukey is a simple post hoc test to run in python
- results compare every pair and tell you if they are different

```
In [177... col_for_data='tree_dbh'
col_to_group='borough'

df_temp=df[[col_for_data,col_to_group]].dropna()

mod = MultiComparison(df_temp[col_for_data], df_temp[col_to_group])

print(mod.tukeyhsd())
```

Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
Bronx	Brooklyn	2.0452	0.0	1.9472	2.1432	True
Bronx	Manhattan	-1.22	0.0	-1.3422	-1.0978	True
Bronx	Queens	2.8642	0.0	2.771	2.9575	True
Bronx	Staten Island	0.7991	0.0	0.6908	0.9074	True
Brooklyn	Manhattan	-3.2652	0.0	-3.3728	-3.1577	True
Brooklyn	Queens	0.819	0.0	0.746	0.8919	True
Brooklyn	Staten Island	-1.2461	0.0	-1.3376	-1.1547	True
Manhattan	Queens	4.0842	0.0	3.981	4.1874	True
Manhattan	Staten Island	2.0191	0.0	1.9021	2.1361	True
Queens	Staten Island	-2.0651	0.0	-2.1515	-1.9788	True

Fun graph! For no use

I got bored and wanted to include cumulative sum on a second axis

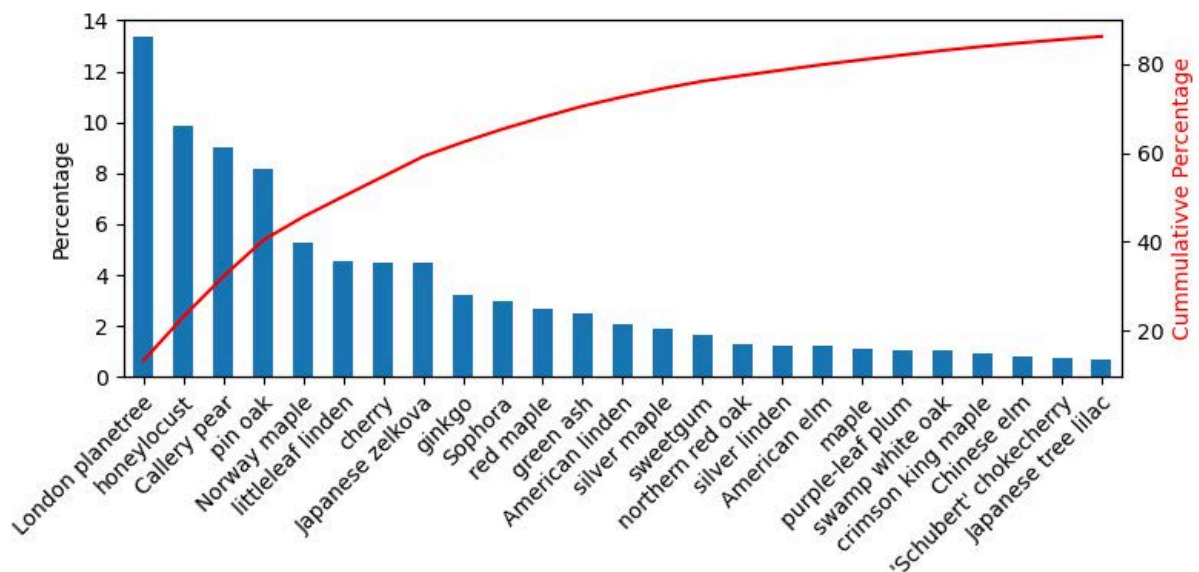
Choose how many tree species you want to plot!

```
In [84]: fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(8,4)
trees_to_plot=25
df_plot=(df['spc_common'].value_counts(normalize=True)*100)
df_plot[0:trees_to_plot].plot.bar(ax=ax)
ax.set_ylabel('Percentage')
ax.set_xlabel('')
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha="right", rotation_mode="anchor")

ax2=ax.twinx()
ax2.plot(np.arange(trees_to_plot), df_plot[0:trees_to_plot].cumsum(), color='red')
ax2.set_ylabel('Cumulative Percentage',color='red')
```

Out[84]: Text(0, 0.5, 'Cumulative Percentage')

Figure



In []: